

Exercise 7: Financial Forecasting

Recursion:

Recursion is a technique where a method calls itself to solve a problem by breaking it into smaller subproblems.

A recursive solution consists of:

Base Case: Condition that stops recursion.

Recursive Case: Method calls itself with a smaller problem.

For financial forecasting, if a value grows at a fixed rate every year:

Future Value(n) = Future Value(n-1) × (1 + growthRate)

Main.java:

```
import java.util.Scanner;
```

```
public class FutureValue {
```

```
    public static double predictValue(double currentValue, double growthRate, int years) {  
        if (years == 0) {  
            return currentValue;  
        }  
    }
```

```
    return predictValue(currentValue, growthRate, years - 1) * (1 + growthRate);  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter current value: ");  
    double currentValue = sc.nextDouble();  
  
    System.out.print("Enter annual growth rate (%): ");  
    double growthRate = sc.nextDouble() / 100;
```

```

        System.out.print("Enter number of years: ");
        int years = sc.nextInt();

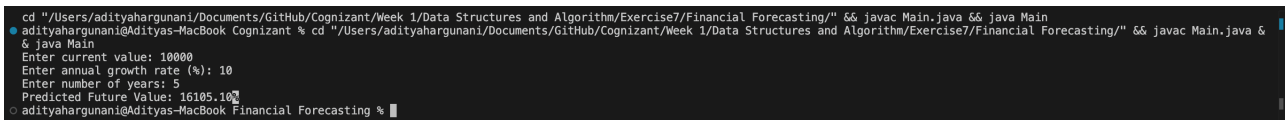
        double futureValue = predictValue(currentValue, growthRate, years);

        System.out.printf("Predicted Future Value: %.2f", futureValue);

        sc.close();
    }
}

```

Output:



```

cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Data Structures and Algorithm/Exercise7/Financial Forecasting/" && javac Main.java && java Main
adityahargunani@Adityas-MacBook Cognizant % cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Data Structures and Algorithm/Exercise7/Financial Forecasting/" && javac Main.java &
& java Main
Enter current value: 10000
Enter annual growth rate (%): 10
Enter number of years: 5
Predicted Future Value: 16105.10
adityahargunani@Adityas-MacBook Financial Forecasting %

```

Time Complexity Analysis:

Recursive function:

predictValue(currentValue, growthRate, years), makes one recursive call for each year.

$$T(n) = T(n-1) + O(1)$$

Therefore,

$$\text{Time Complexity} = O(n)$$

where n is the number of years.

Optimization

The recursive solution creates a new stack frame for every year.

A more efficient mathematical formula is:

Future Value = Present Value $\times$ (1 + growthRate)^{years}

Optimized Java Code:

```
double futureValue = currentValue * Math.pow(1 + growthRate, years);
```

Time Complexity for Optimized Code: **$O(\log n)$**